先找到 disp2handler,跑个 Trace 统计影响 EIP 指令的地址数即可,发现 disp2handler。

0064F6D8 <safe | E8 ECA4FEFF | call safengine 授权对抗 ai.639BC9

{

HITS KIND TARGETS ADDR INSN

---

310 ret 21 0x82BA1E ret 0x64 disp2handler

1 ret 1 0x82AA51 ret 0x30

1 jmp indirect 1 0x639763 jmp [ds: + 0x63B044]

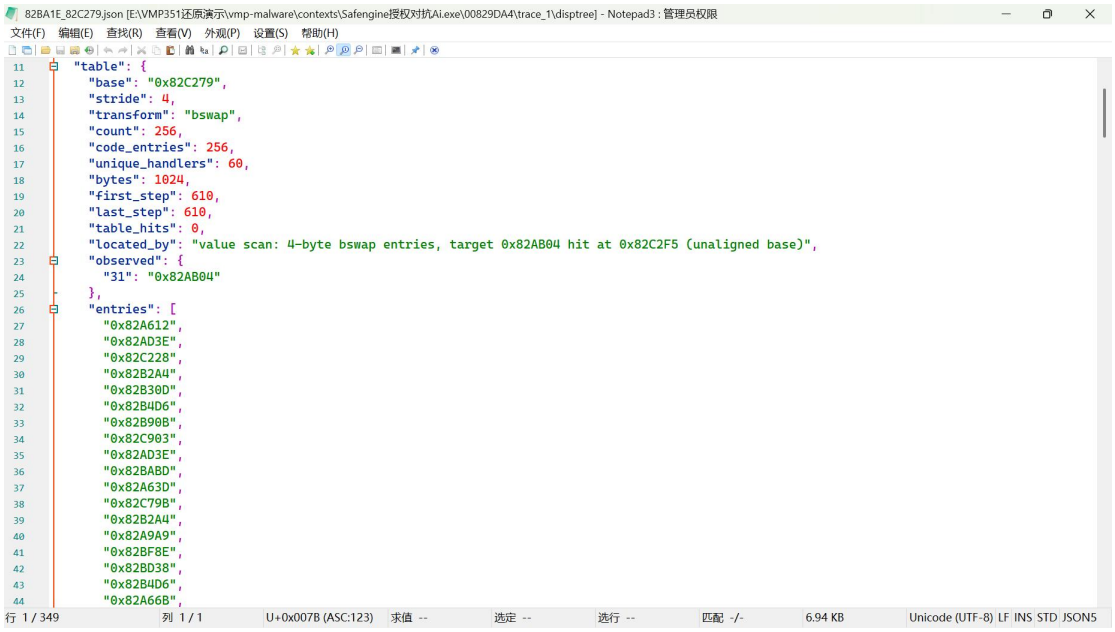
}

来个 workflow 分析 Trace 根据 disp2handler 做污点分析回溯 handler table，拿到特征

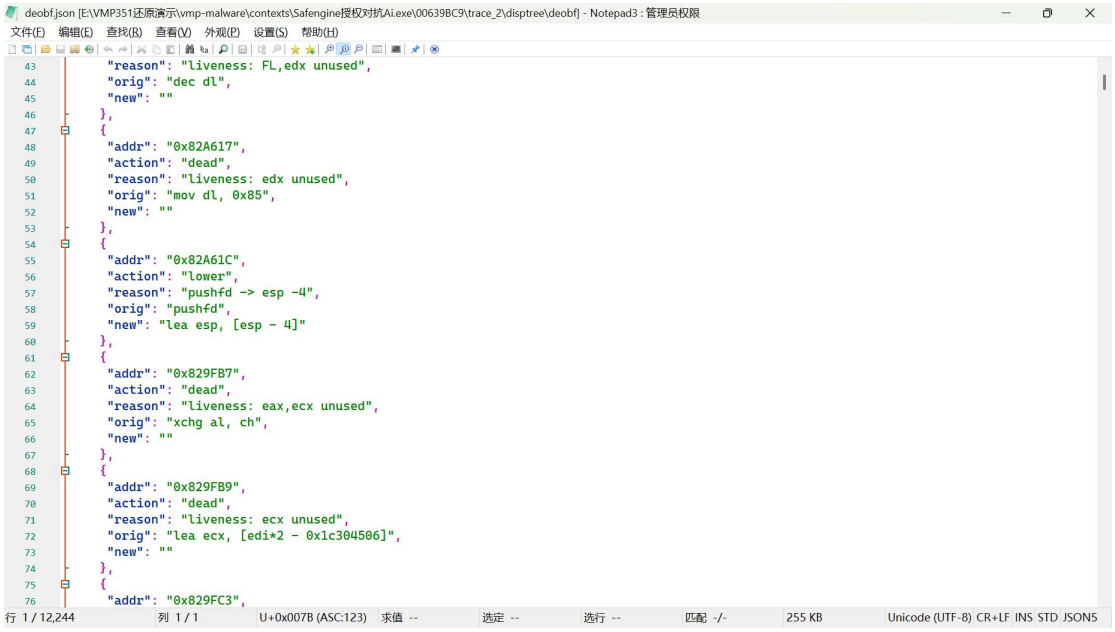
ret 型（Safengine 2.x）：`ret imm16`，handler 由更早的

`mov reg,[disp32 + idx*N]` 取出、变换后写回栈顶再弹出

拿到 handler 表



Handler 有混淆，来个 workflow，分析 Trace 当中的混淆特征，基于活跃变量分析通过 Nop、简单指令替换的方式削弱死代码去混淆，有些 pushfd、popfd 基于栈的混淆要求 lowering 到简单指令，还原回正常不依赖栈的汇编。

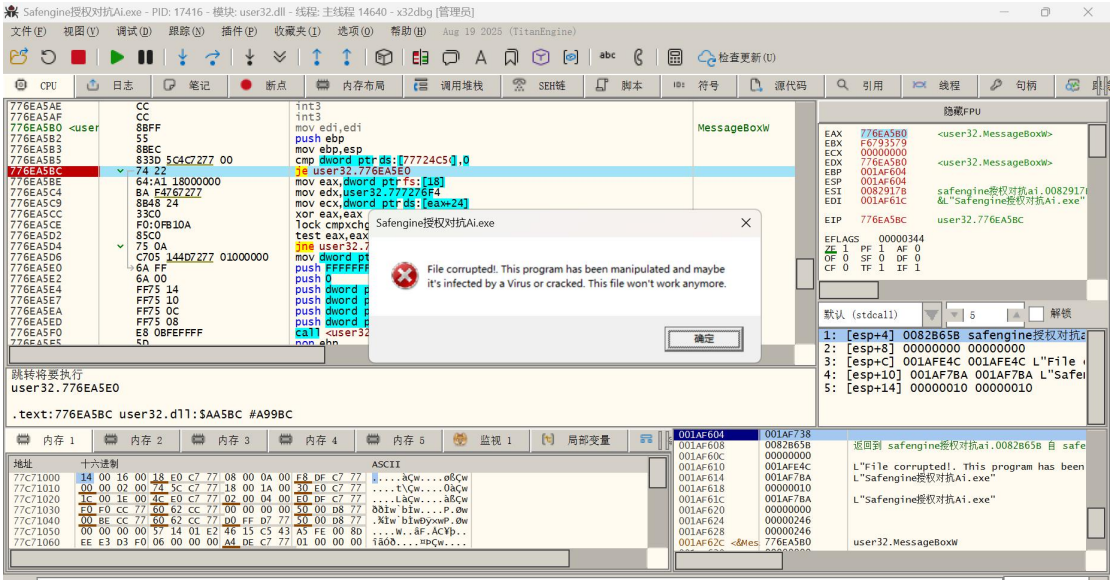


接下来个 workflow 分析 Safengine 的虚拟机架构，根据去混淆的汇编 CFG 分析 Safengine 虚拟机的架构，是基于堆的虚拟机还是栈的虚拟机，是 CISC 还是 RISC。

结果

Safengine 2.x 是栈式虚拟机（stack machine），指令集是 CISC 语义、RISC 实现。Handler 表有 256 个 handler，去重后 60 个，其中有 3 种类型的 vRet 走自己的 handler 尾部退出虚拟机。其他 handler 尾部都会回到虚拟机循环里重新调度。相比于 VMProtect3.5，Safengine 2.x 的 opcode 是指令变长的。

随便下个断点触发 File corrupted，文件被修改，说明有内存校验 vChecker。



vChecker 在 0x82AE17，将其抽象为函数，参数 1 为校验地址，参数 2 为校验长度。

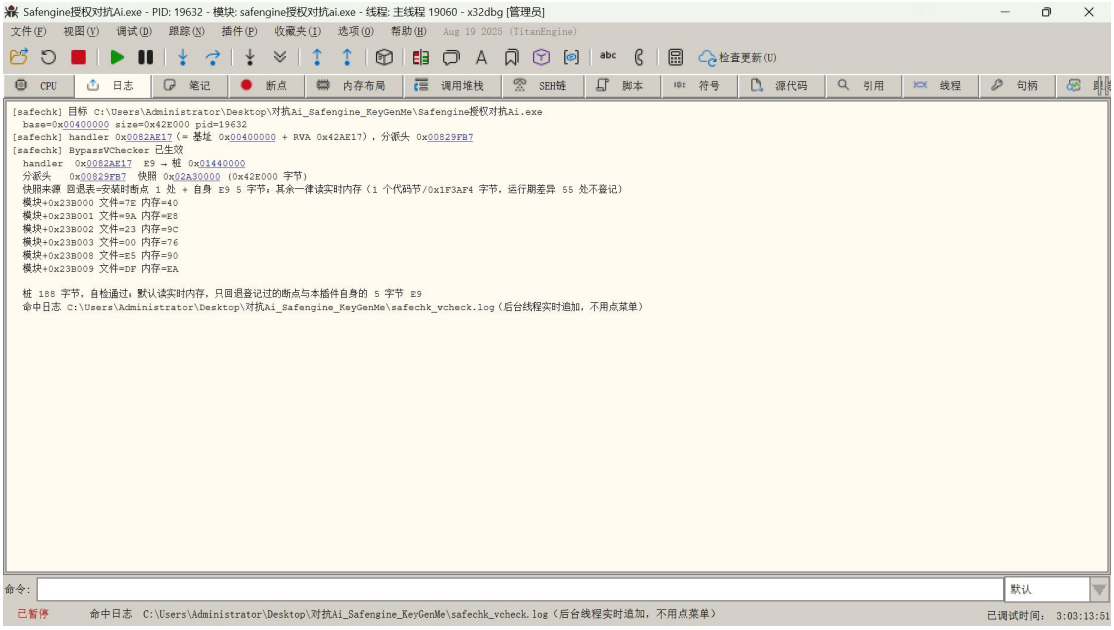
```
**vRdtsc · 0x82AE70 · slots 19,127,157,175,240,251 — RDTSC**
```
rdtsc ; sub ebp,8 ; mov [ebp],edx ; mov [ebp+4],eax
```
时间戳高低 32 位压栈。栈：-8。

**vCheck · 0x82AE17 · slots 57,104,156,203,229,236 — 内存校验循环（唯一带内层循环的 handler）**
```
mov edx,[ebp] ; add ebp,4 ; xor eax,eax
loop: mov ecx,eax ; shl eax,7 ; shr ecx,0x19 ; or eax,ecx ; xor al,[edx] ; ... dec [ebp] ; jne loop
mov [ebp],eax
```
对 `[edx]` 指向的缓冲做"滚动异或/位混合"直到计数到 0 — 字符串/校验和/解密循环（`rep` 类语义的软件实现）。

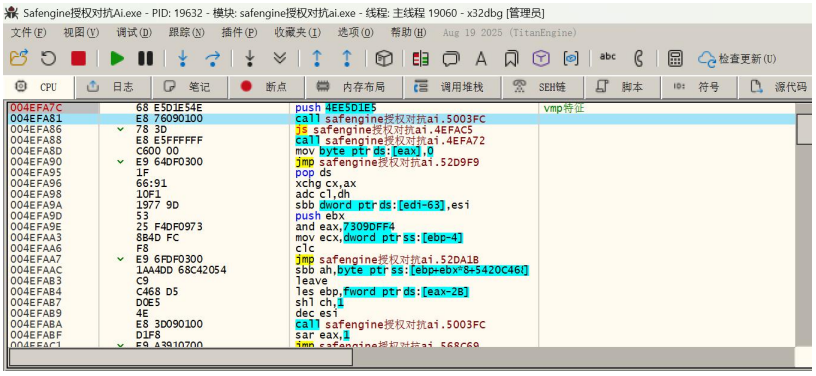
### 控制流出口（3 个）

**vRet · 0x82B4D6 · slots 5,16,85,183 — VM 出口（恢复 8 个寄存器）**
```

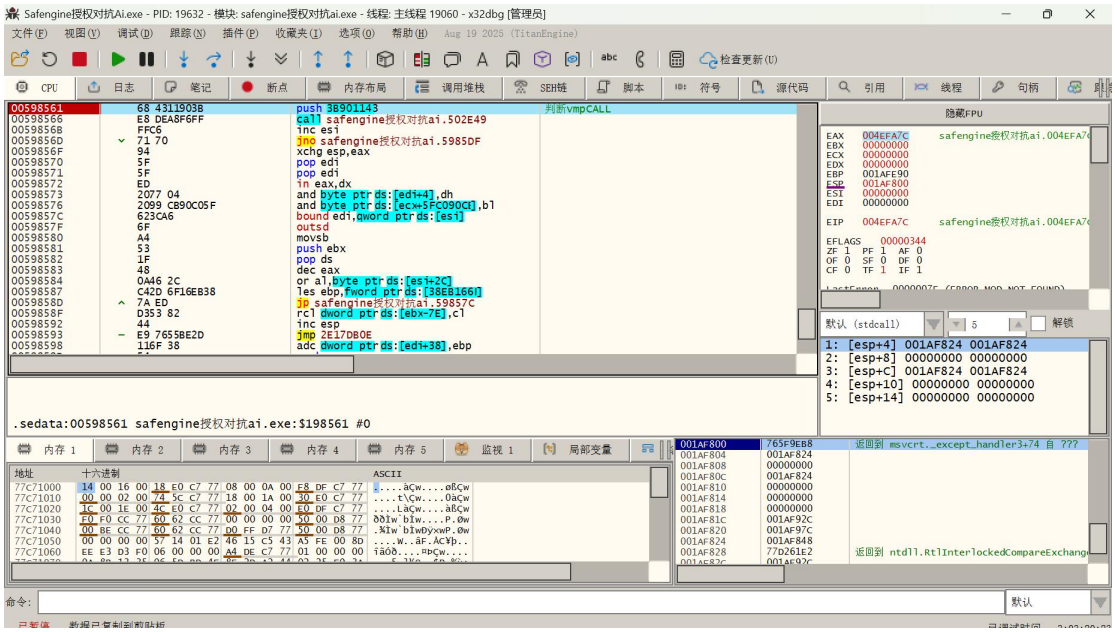
来个 workflow，编写内存注入 dll 做 bypassSafengine2.xVChecker，忽略掉我们的断点，将 hash 值 hook 为预期的即可。预期的 hash 值通过 unicorn 模拟和我们实际的 hookVChecker 修改结果做对比，如不一致调试。最终达到 bypass 内存校验的效果。

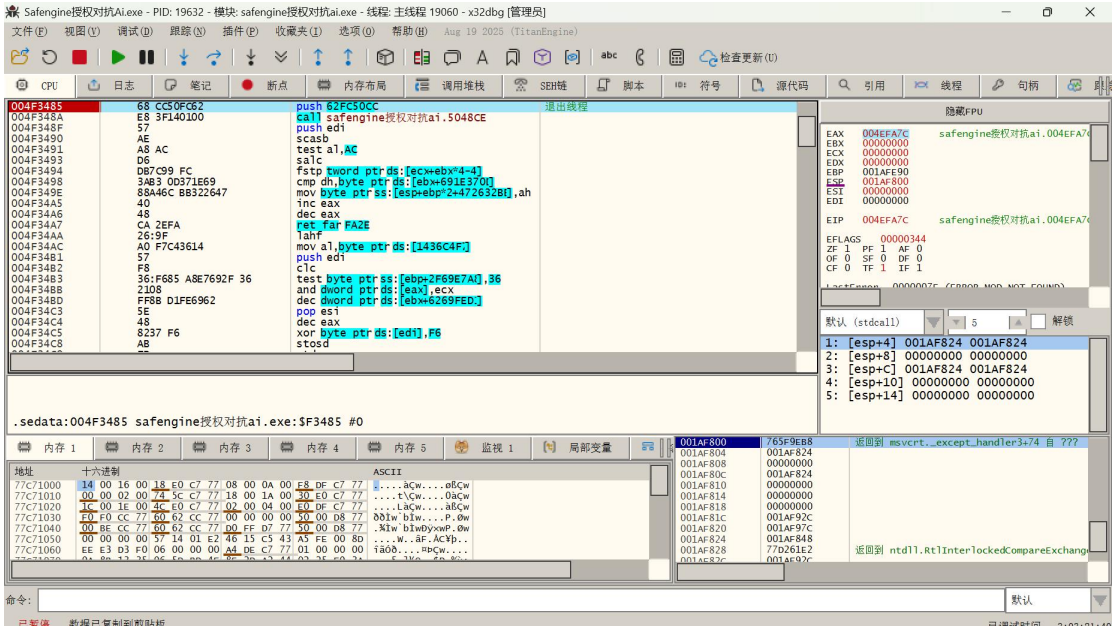


接下去就是跟踪 VM 了，通过不断地侧信道分析 vRet、vCpuid，最终定位到 004EFA7C 这个地址。同时发现 vmp 特征，看样子作者在 safengine 的基础上套了一层 vmprotect 的保护壳，很强大。



继续跟踪 VM，到了 00598561，这是个判断 CALL，判断用户机器码与预期的机器码是否一致。如果判断失败会启动一个退出线程，参数地址为 004F3485，也是一个被 VMP 保护的函数。

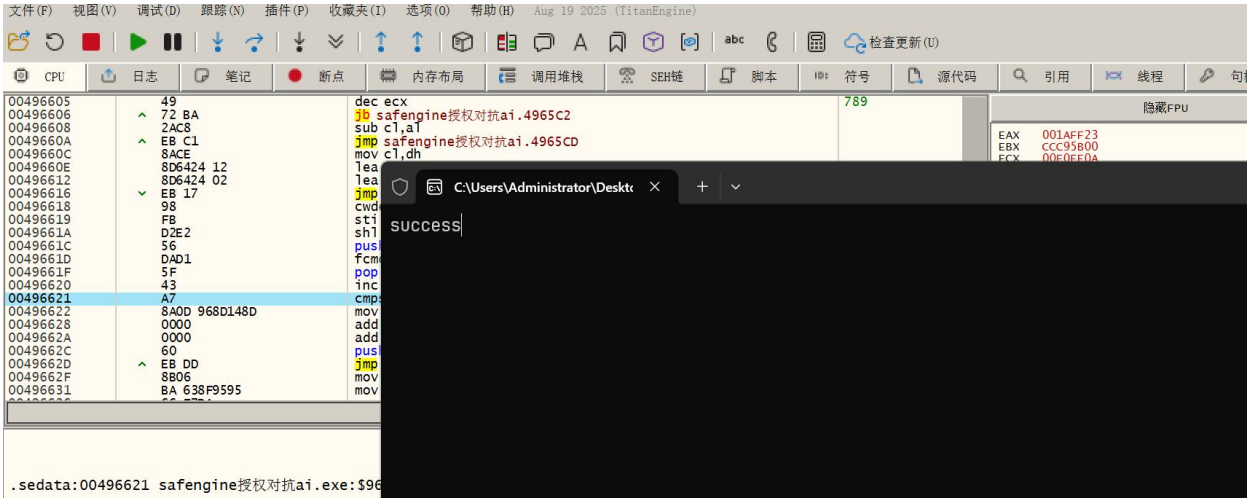




通过不断的跟踪，发现 Safengine 自行重载了 dll 进行 api 的函数调用，总之跟了一堆的 VM 函数调用。最终定位到计算出 base64 解码后机器码 machinecode 的函数地址，这块没有混淆，类似一个 shellcode。在 SystemBiosVersion 调用后进行 patch 机器码即可，要 patch 俩次，因为程序有两次获取机器码进行比较的操作。这可能也是作者魔改后的效果，总之，最终 patch 结果如下。

程序默认弹出机器码“1+LKtQAIEABTBgoAeqbPxVAJBgeiZysuFallCg==”，注意到 Fail，这说明用户无法通过搜索该机器码的 base64 解码内存进行回溯机器码的计算，因为仅在弹窗时显示该内容。

2026 年 11 月 10 日


文件(F) 视图(V) 调试(D) 跟踪(N) 插件(P) 收藏夹(I) 选项(O) 帮助(H) Aug 19 2025 (TitanEngine)
检查更新(U)
CPU 日志 笔记 断点 内存布局 调用堆栈 SEH链 脚本 符号 源代码 引用 线程
00496605 49 dec ecx
00496606 72 BA jb safengine授权对抗ai.4965C2
00496608 2AC8 sub cl,al
0049660A EB C1 jmp safengine授权对抗ai.4965CD
0049660C 8ACE mov cl,dh
0049660E 8D6424 12 lea
00496612 8D6424 02 lea
00496616 EB 17 jmp
00496618 98 cwd
00496619 FB sti
0049661A D2E2 shl
0049661C 56 push
0049661D DAD1 fcm
0049661F 5F pop
00496620 43 inc
00496621 A7 cmp
00496622 8A0D 968D148D mov
00496628 0000 add
0049662A 0000 add
0049662C 60 push
0049662D EB DD jmp
0049662F 8B06 mov
00496631 BA 638F9595 mov
789
隐藏FPU
EAX 001AFF23
EBX CCC95B00
C:\Users\Administrator\Deskt
success
.sedata:00496621 safengine授权对抗ai.exe:$96